

# Using Design Patterns

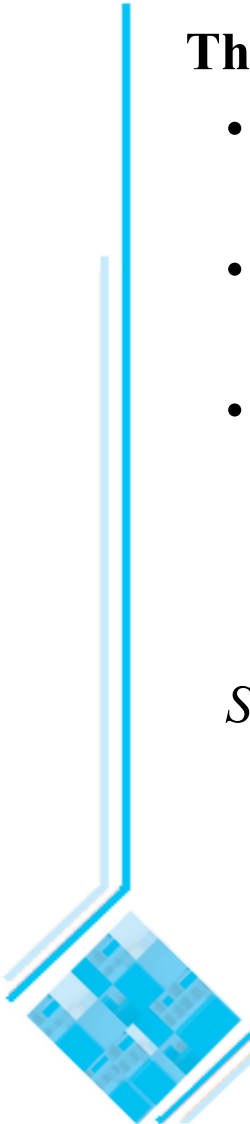
Adapted from:  
Timothy Lethbridge and Robert Laganieri,  
Object-Oriented Software Engineering –  
Practical Software Development using UML and Java, 2005  
(chapter 6)

# 6.1 Introduction to Patterns

**The recurring aspects of designs are called *design patterns*.**

- A *pattern* is the outline of a reusable solution to a commonly occurring problem encountered in a particular context
- Many of them have been systematically documented for all software developers to use
- A good pattern should
  - Be as general as possible
  - Contain a solution that has been proven to effectively solve the problem in the indicated context.

*Studying patterns is an effective way to learn from the experience of others*



# Introduction to Patterns

- The notion of patterns was developed in an engineering context by the architect and design theorist **Christopher Alexander**
  - His work has influenced Computer Science
- Design patterns became popular in Software Engineering after the publication of the book:

[GoF94] E. Gamma, R. Helm, R. Johnson and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. (The four authors of this book are known as the “**Gang of Four**”, abbreviated as “**GoF**”)

- The GoF book [GoF94] presents a catalog of 23 design patterns .

# Pattern description

## **Context:**

- The general situation in which the pattern applies

## **Problem:**

—A short sentence or two raising the main difficulty.

## **Forces:**

- The issues or concerns to consider when solving the problem

## **Solution:**

- The recommended way to solve the problem in the given context.  
—‘to balance the forces’

## **Antipatterns: (Optional)**

- Solutions that are inferior or do not work in this context.

## **Related patterns: (Optional)**

- Patterns that are similar to this pattern.

## **References:**

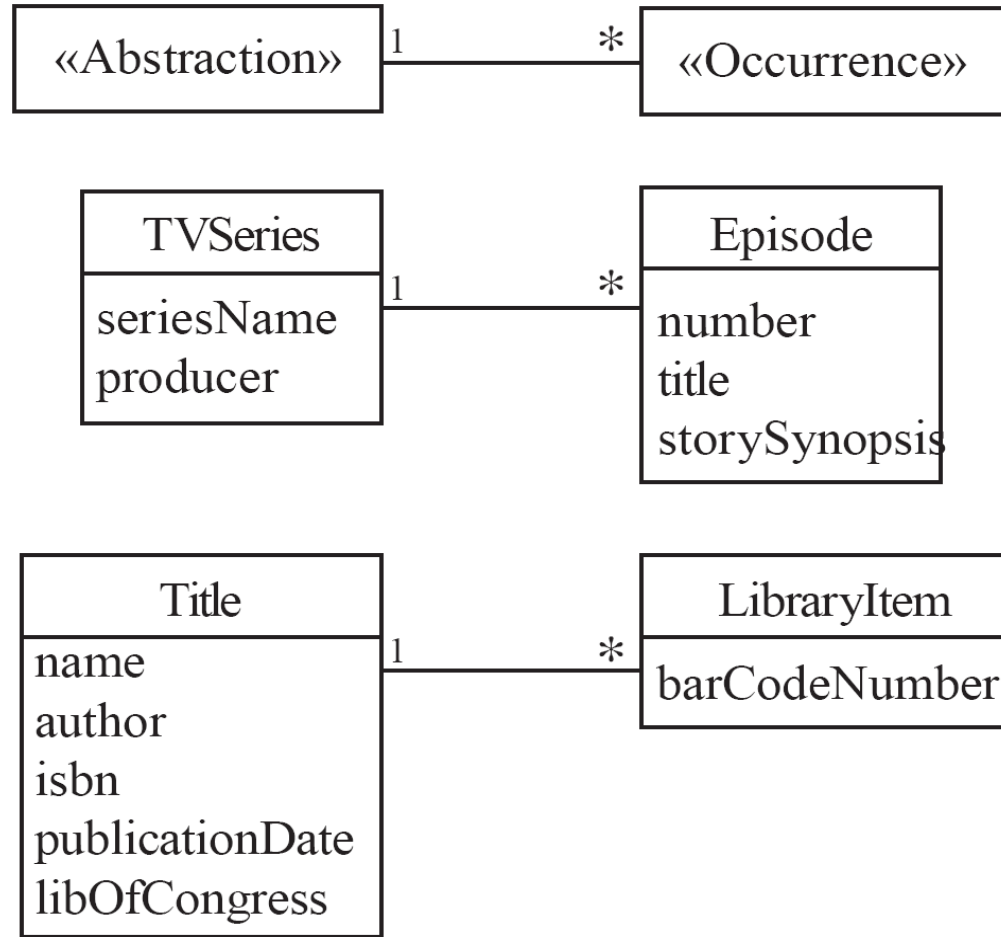
- Who developed or inspired the pattern.

## 6.2 The Abstraction-Occurrence Pattern

- **Context:**
  - Often in a domain model you find a set of related objects (*occurrences*).
  - The members of such a set share common information
    - but also differ from each other in important ways.
- **Problem:**
  - What is the best way to represent such sets of occurrences in a class diagram?
- **Forces:**
  - You want to represent the members of each set of occurrences without duplicating the common information
- **References:**
  - Abstraction-Occurrence is a generalization of the Title-Item pattern [EP00]

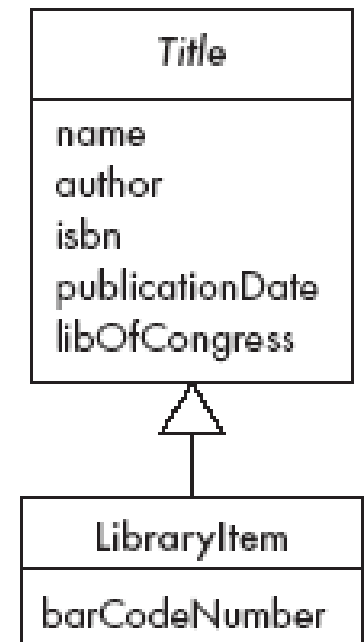
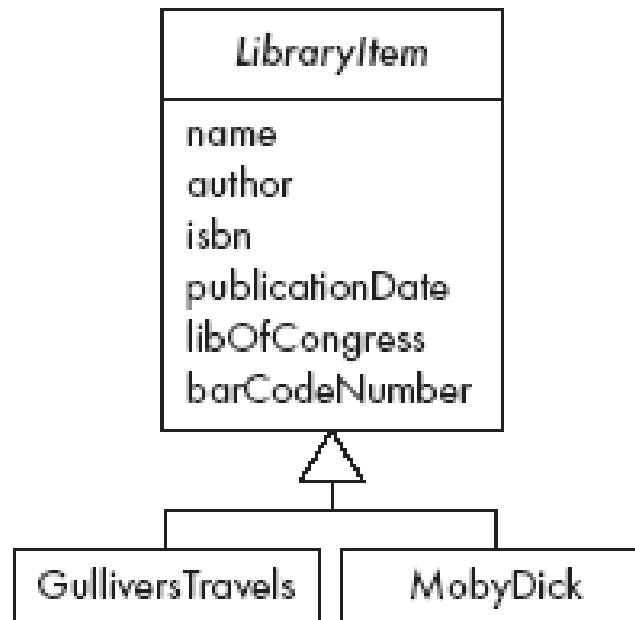
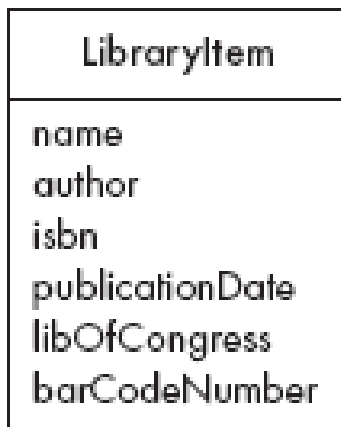
# Abstraction-Occurrence

- *Solution:*



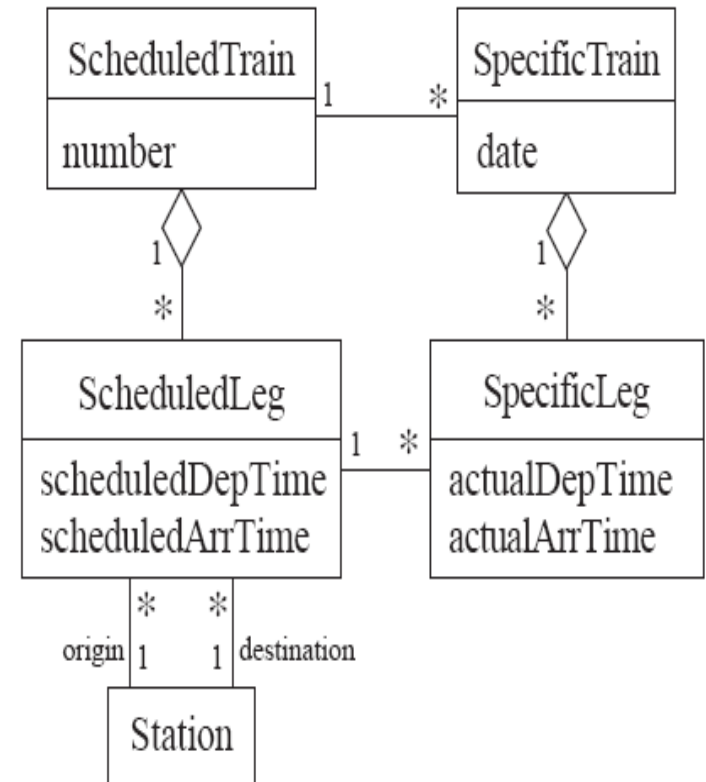
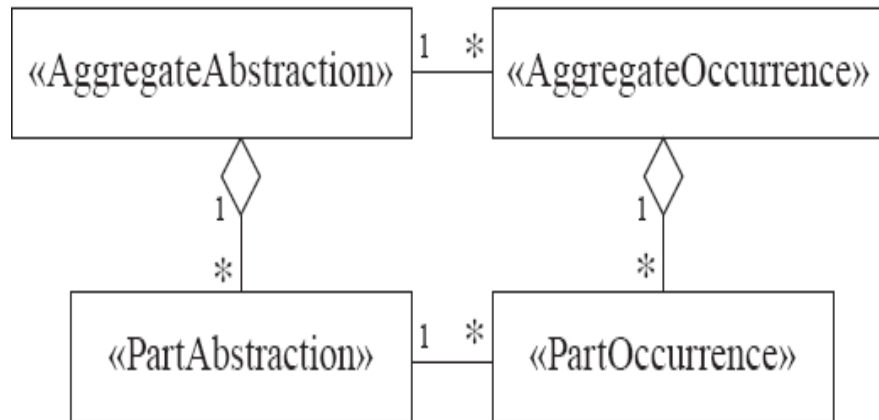
# Abstraction-Occurrence

## Antipatterns:



# Abstraction-Occurrence

## Square variant





## 6.3 The General Hierarchy Pattern

- **Context:**

- Objects in a hierarchy can have one or more objects above them (superiors), and one or more objects below them (subordinates).
- Some objects cannot have any subordinates

- **Problem:**

- How do you represent a hierarchy of objects, in which some objects cannot have subordinates?

- **Forces:**

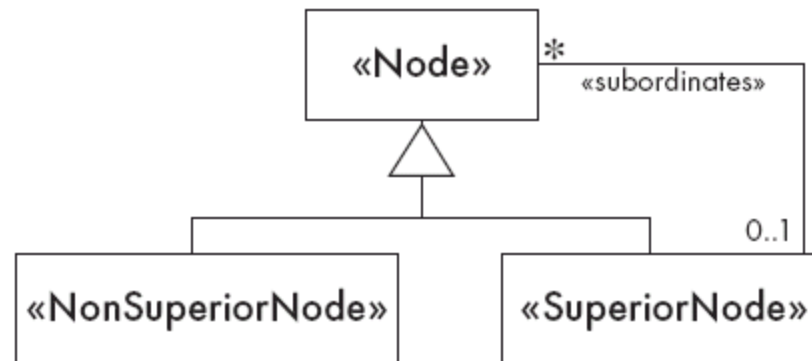
- You want a flexible way of representing the hierarchy
  - that prevents certain objects from having subordinates
- All the objects have many common properties and operations

- **References:**

- The Composite pattern described in [GoF94] is a specialization of the General Hierarchy pattern [LL05] (in the Composite pattern the association between an inner node and its subordinates is an aggregation). [GoF94] also includes Composite in the related patterns list of the Interpreter pattern.

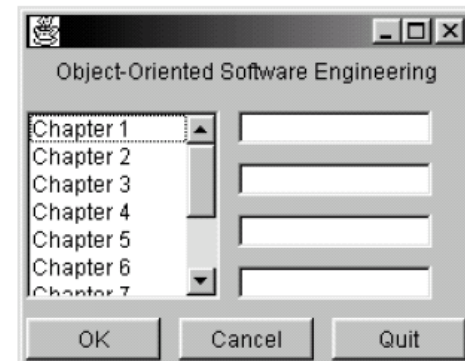
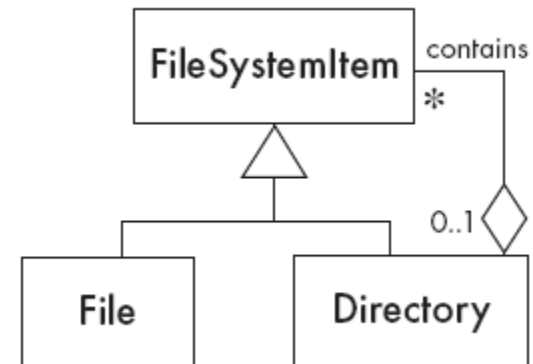
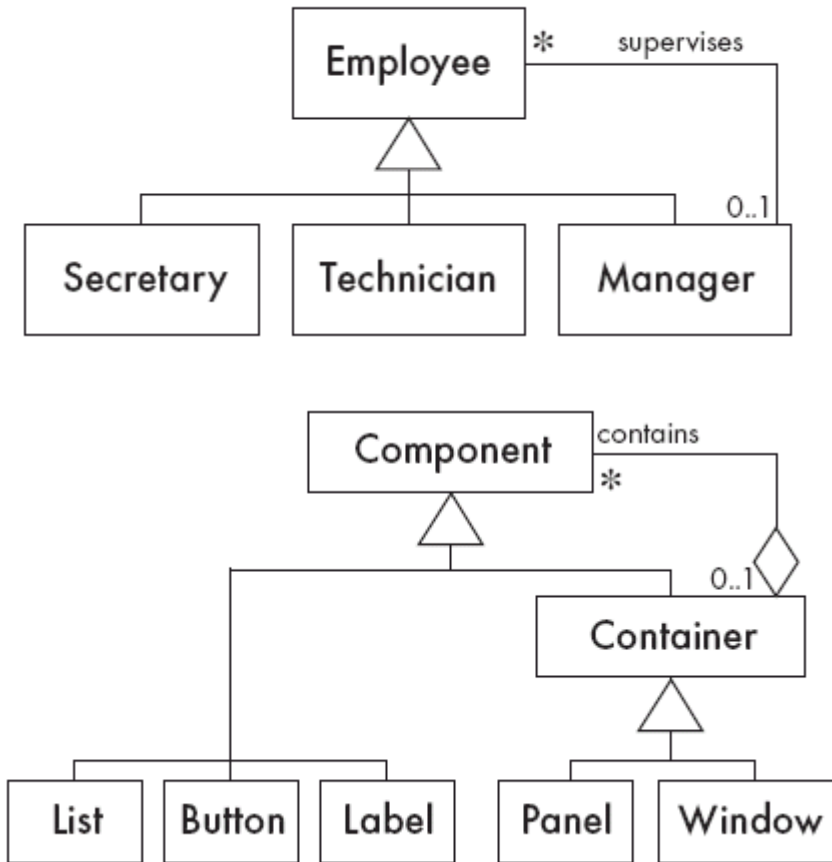
# General Hierarchy

- *Solution:*



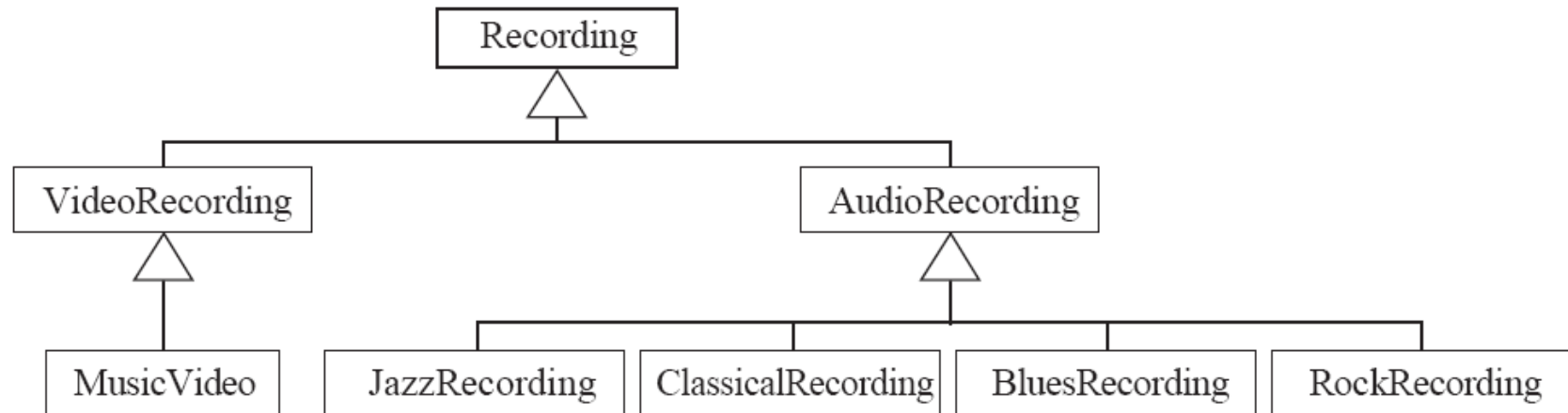
# General Hierarchy

- *Solution:*



# General Hierarchy

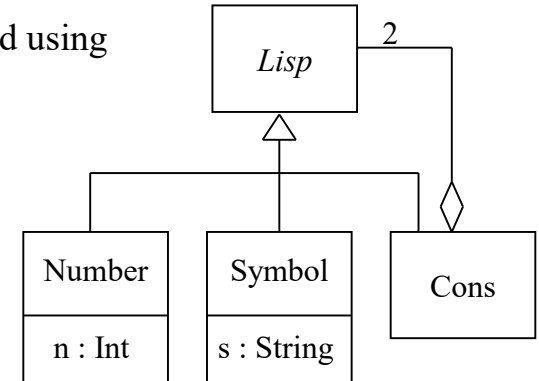
**Antipattern:**



# Implementation Notes

- Consider a **Java implementation** of the Lisp data structure (designed using the **Composite pattern**):

```
abstract class Lisp { ... }  
class Number extends Lisp { int n; ... }  
class Symbol extends Lisp { String s; ... }  
class Cons extends Lisp {  
    Lisp car;  
    Lisp cdr;  
    ...  
}
```



- In Scala you can implement Composite as above (in Java style) [Nik16]. However, a **Scala implementation** of **Composite** could also use **case classes**:

```
abstract class Lisp  
case class Number (n: Int) extends Lisp  
case class Symbol (s: String) extends Lisp  
case class Cons (car: Lisp, cdr: Lisp) extends Lisp
```

# Implementation Notes

abstract class Lisp

case class Number (n: Int) extends Lisp

case class Symbol (s: String) extends Lisp

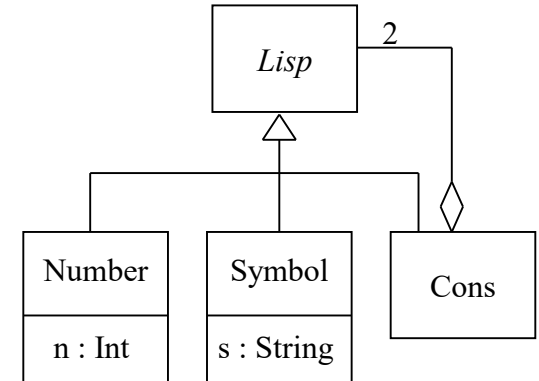
case class Cons (car: Lisp,cdr: Lisp) extends Lisp

- **Scala case classes** support **pattern matching**. The following Scala function implements a flattening operation:

```
def flat (lisp: Lisp): Lisp = lisp match {  
  case Cons (Number(n),cdr)      => Cons (Number (n),flat(cdr))  
  case Cons (Symbol(s),cdr)      => Cons (Symbol (s),flat(cdr))  
  case Cons (Cons(caar,cdar),cdr) => flat (Cons (caar,Cons (cdar,cdr)))  
  case lisp                      => lisp  
}
```

```
flat(Cons (Cons (Number(1),Number (2)),null)) →  
  Cons (Number(1),Cons(Number (2),null))
```

- The **Scala solution** is **concise**. Imagine implementing this function (flat) in Java style, using if statements !



## 6.4 The Player-Role Pattern

- ***Context:***

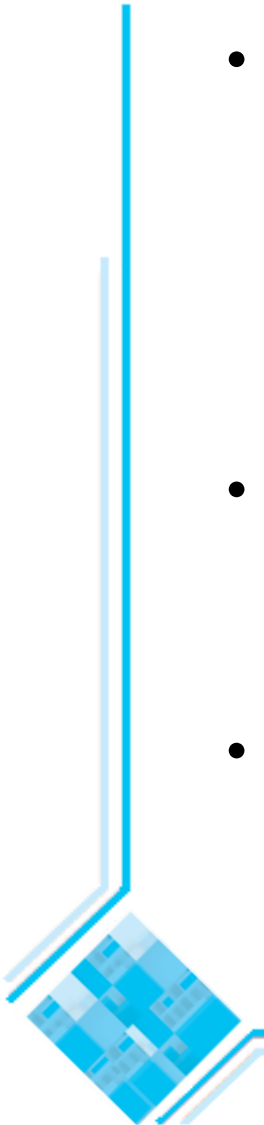
- A *role* is a particular set of properties associated with an object in a particular context.
- An object may *play* different roles in different contexts.

- ***Problem:***

- How do you best model players and roles so that a player can change roles or possess multiple roles?

- ***References:***

- The Player-Role pattern appears in [Rum91]

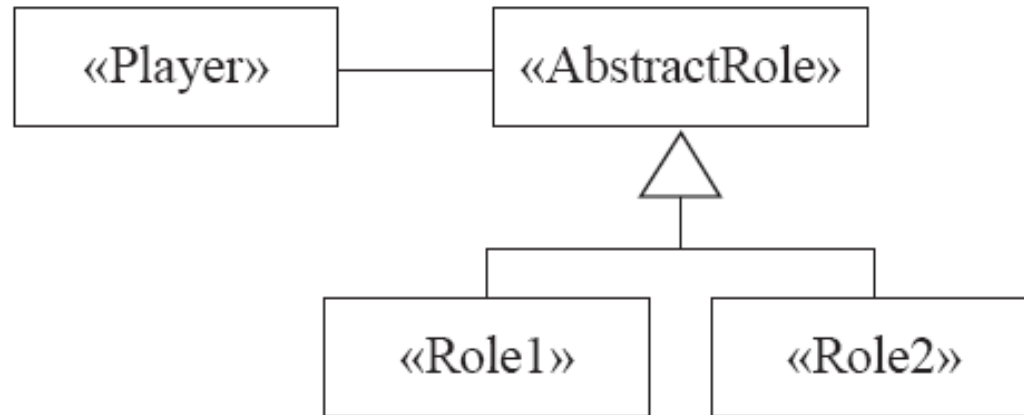


# Player-Role

- ***Forces:***

- It is desirable to improve encapsulation by capturing the information associated with each separate role in a class.
- You want to avoid multiple inheritance.
- You cannot allow an instance to change class

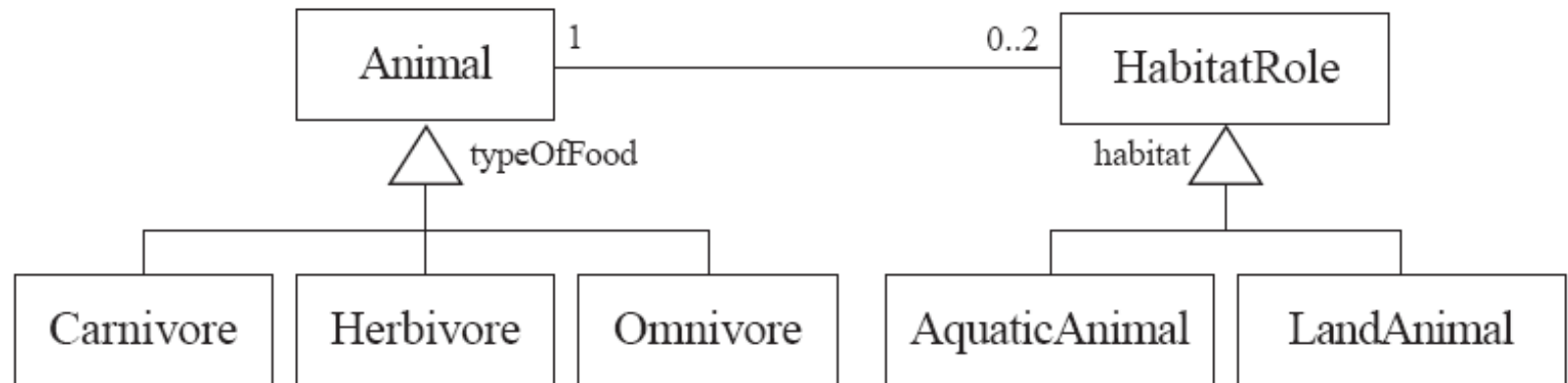
- ***Solution:***





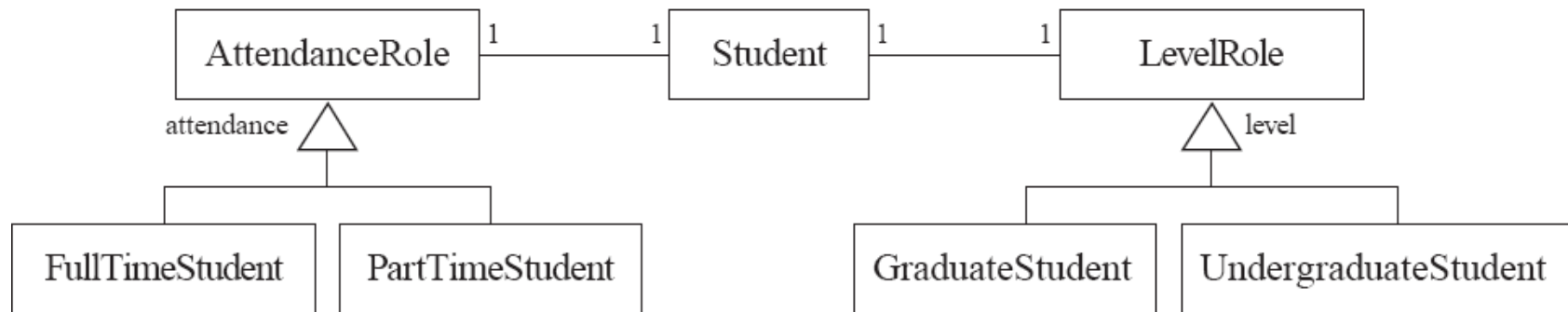
# Player-Role

## Example 1:



# Player-Role

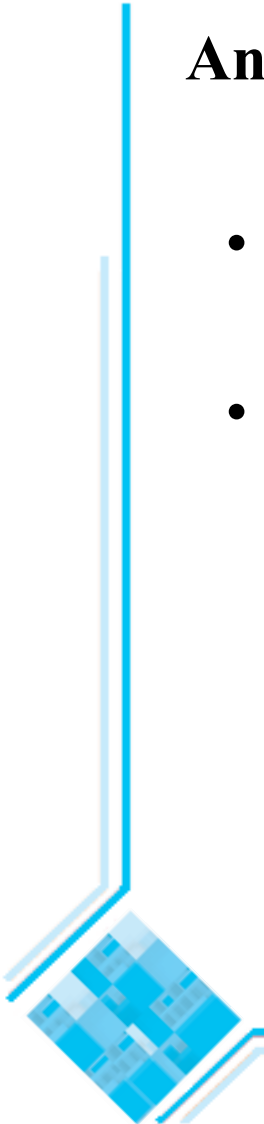
## Example 2:



# Player-Role

## Antipatterns:

- Merge all the properties and behaviours into a single «Player» class and not have «Role» classes at all.
- Create roles as subclasses of the «Player» class.



## 6.5 The Singleton Pattern

- ***Context:***

- It is very common to find classes for which only one instance should exist (*singleton*)

- ***Problem:***

- How do you ensure that it is never possible to create more than one instance of a singleton class?

- ***Forces:***

- The use of a public constructor cannot guarantee that no more than one instance will be created.

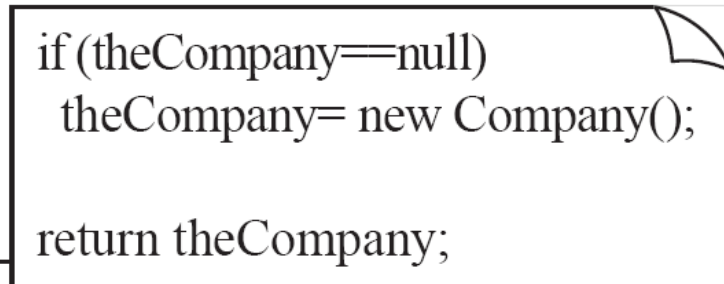
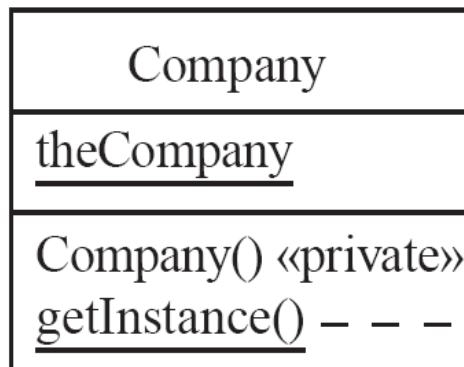
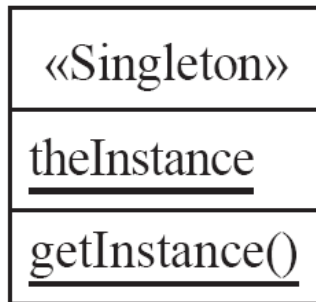
- The singleton instance must also be accessible to all classes that require it

- ***References:***

- Singleton is one of the GoF patterns

# Singleton

- ***Solution:***



## 6.6 The Observer Pattern

- ***Context:***

- When an association is created between two classes, the code for the classes becomes inseparable.
- If you want to reuse one class, then you also have to reuse the other.

- ***Problem:***

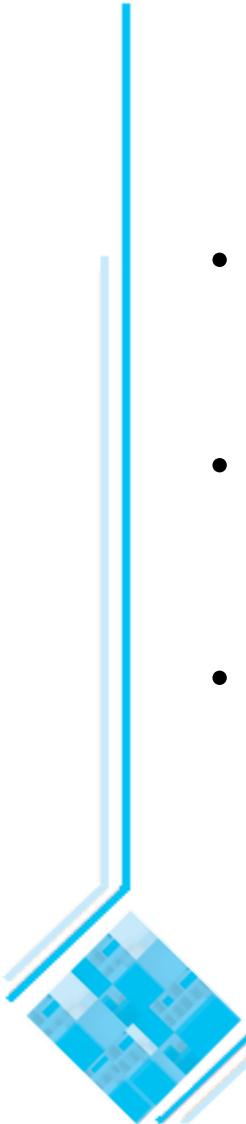
- How do you reduce the interconnection between classes, especially between classes that belong to different modules or subsystems?

- ***Forces:***

- You want to maximize the flexibility of the system to the greatest extent possible

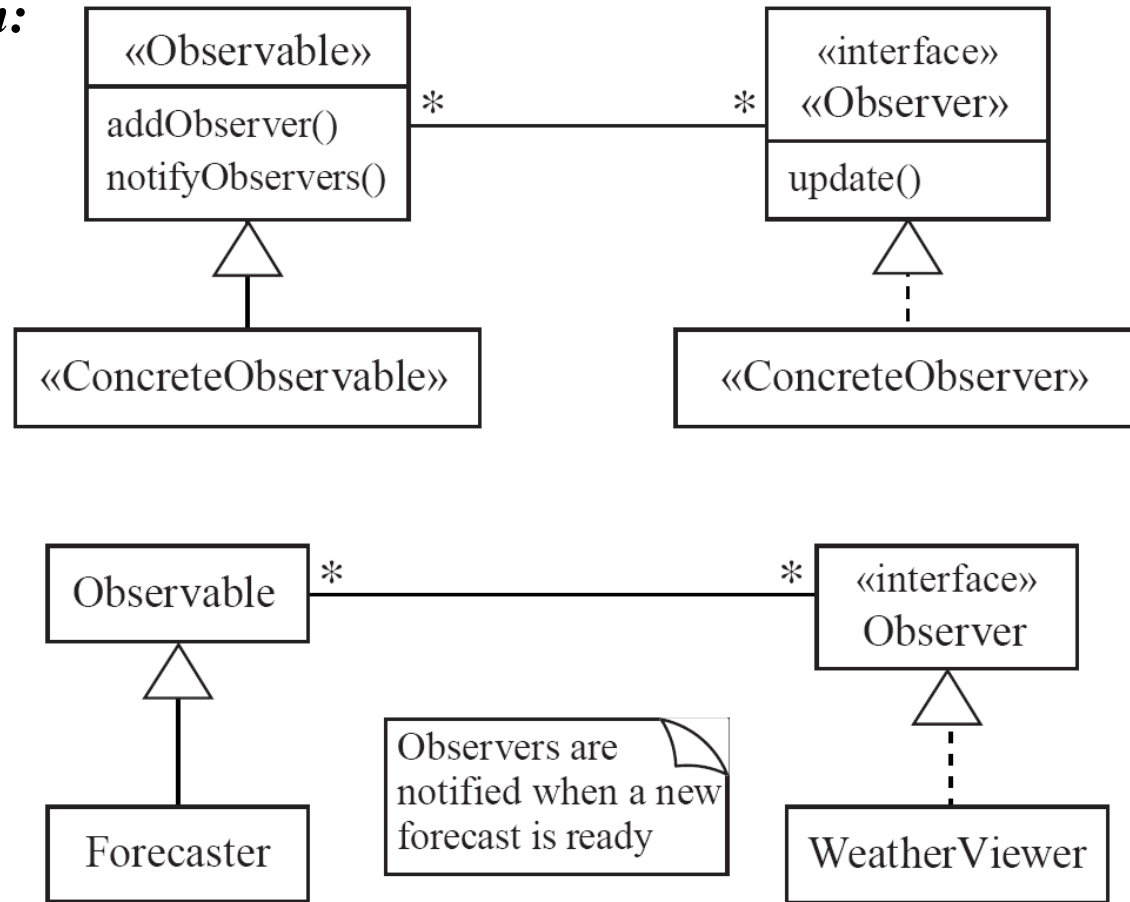
- ***References:***

- Observer is one of the GoF patterns



# Observer

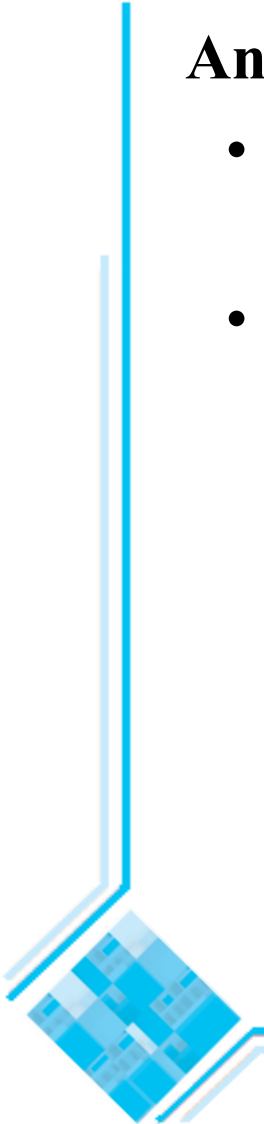
- **Solution:**



# Observer

## Antipatterns:

- Connect an observer directly to an observable so that they both have references to each other.
- Make the observers *subclasses* of the observable.





## 6.7 The Delegation Pattern

- ***Context:***

- You are designing a method in a class
- You realize that another class has a method which provides the required service
- Inheritance is not appropriate
  - E.g. because the isa rule does not apply

- ***Problem:***

- How can you most effectively make use of a method that already exists in the other class?

- ***Forces:***

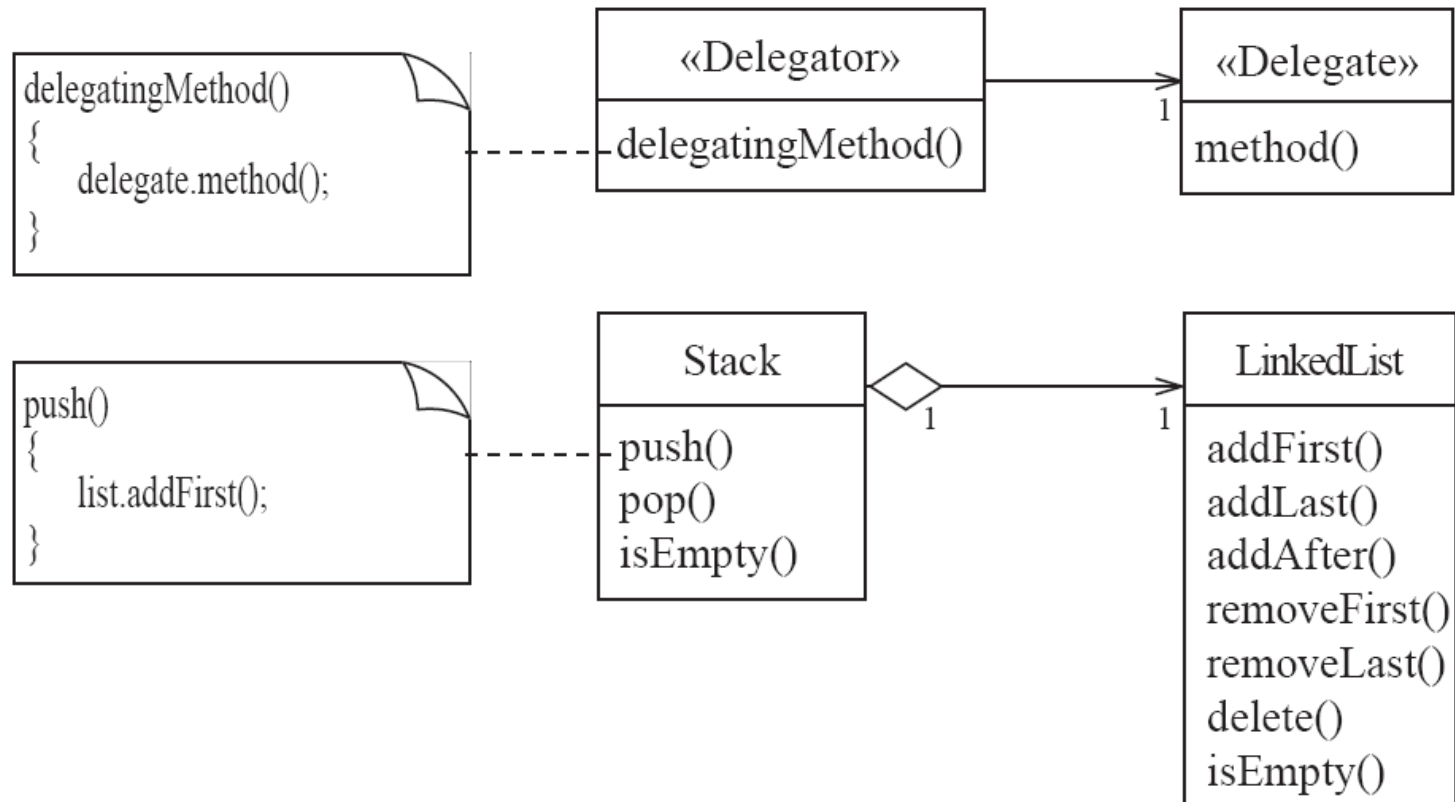
- You want to minimize development cost by reusing methods

- ***References:***

- The Delegation pattern is mentioned in [Gra02]

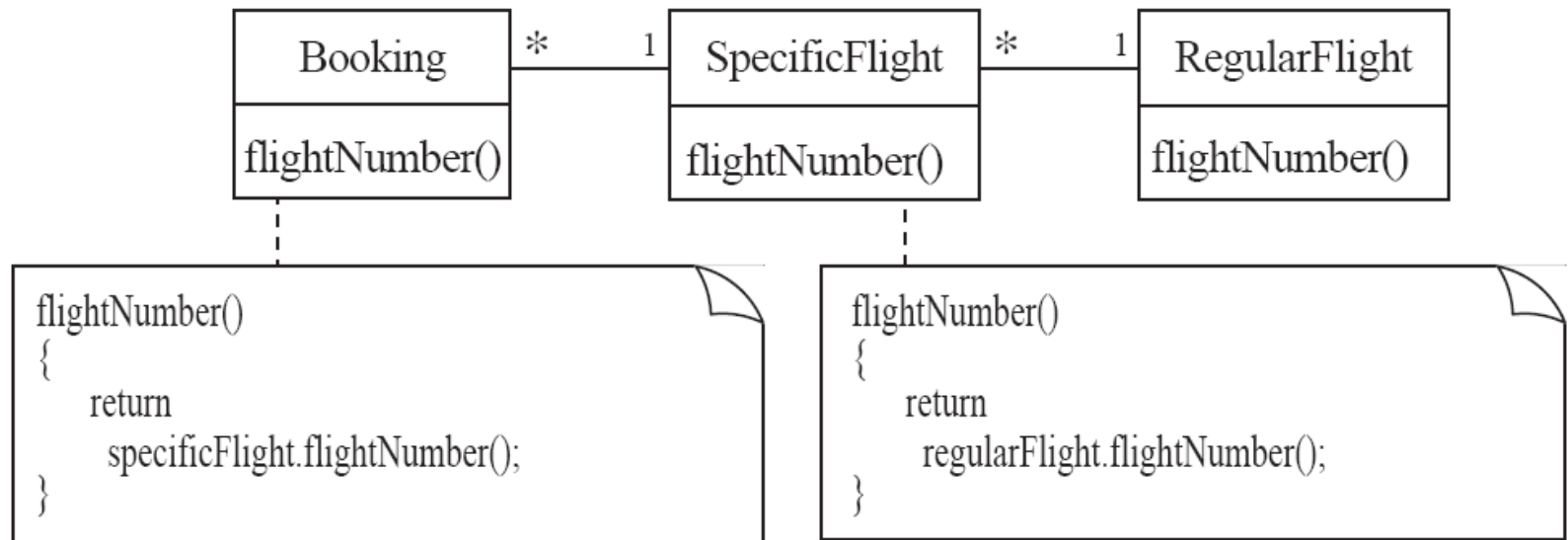
# Delegation

- ***Solution:***



# Delegation

## Example:



# Delegation

## Antipatterns

- Overuse generalization and *inherit* the method that is to be reused
- Instead of creating a *single* method in the «Delegator» that does nothing other than call a method in the «Delegate»
  - consider having many different methods in the «Delegator» call the delegate's method
- Access non-neighboring classes

```
return specificFlight.regularFlight.flightNumber();

return getRegularFlight().flightNumber();
```

## 6.8 The Adapter Pattern

- **Context:**

- You are building an inheritance hierarchy and want to incorporate into it an existing class.
- The reused (existing) class is also often already part of its own inheritance hierarchy.

- **Problem:**

- How to obtain the power of polymorphism when reusing a class whose methods
  - have the same function
  - but *not* the same signatureas the other methods in the hierarchy?

- **Forces:**

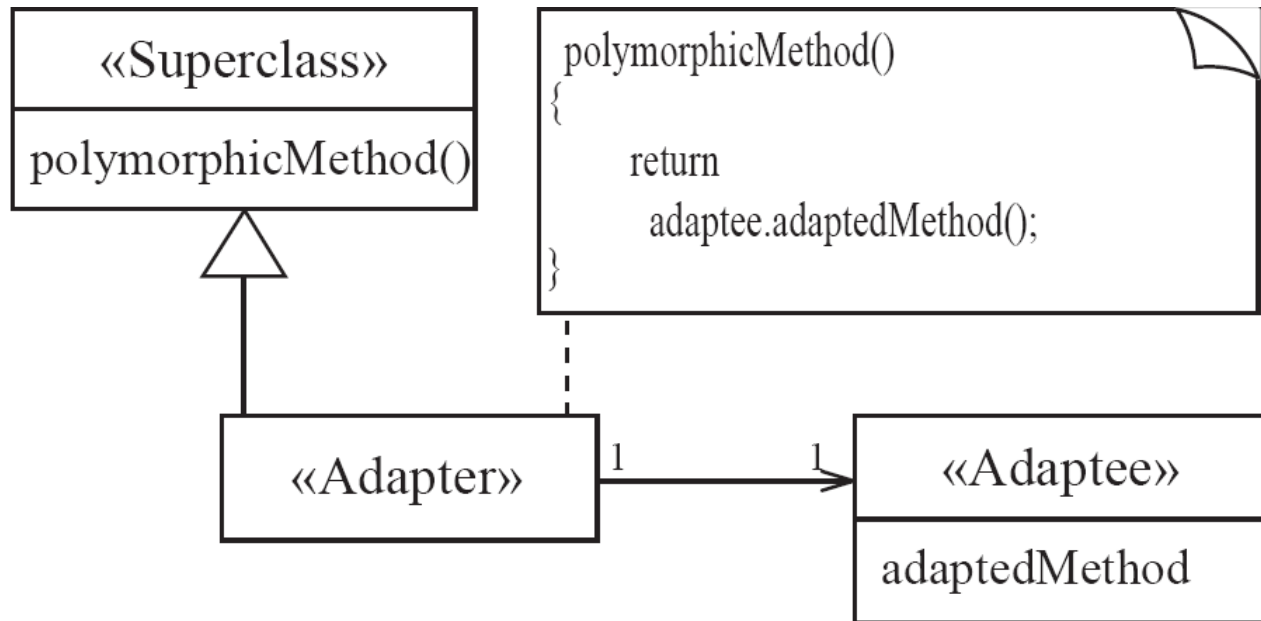
- You do not have access to multiple inheritance or you do not want to use it.

- **References:**

- Adapter is one of the GoF patterns

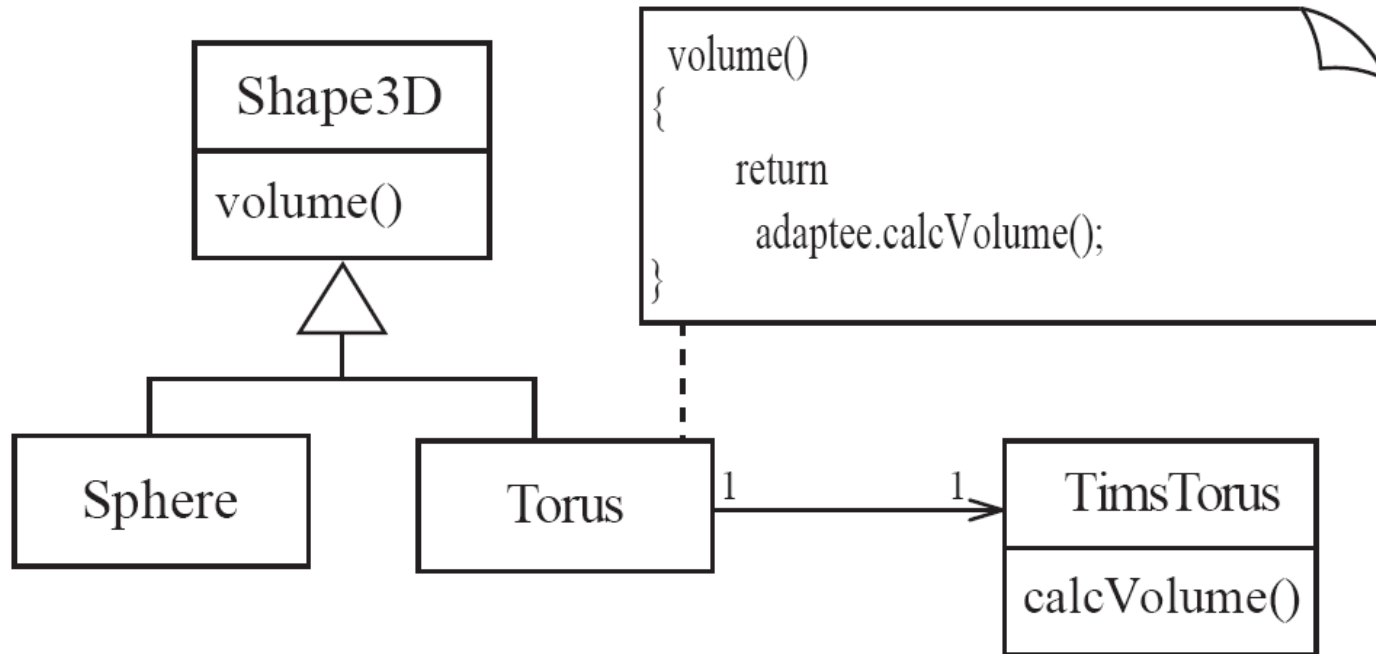
# Adapter

- ***Solution:***



# Adapter

## Example:



## 6.9 The Façade Pattern

- **Context:**

- Often, an application contains several complex packages.
- A programmer working with such packages has to manipulate many different classes

- **Problem:**

- How do you simplify the view that programmers have of a complex package?

- **Forces:**

- It is hard for a programmer to understand and use an entire subsystem
- If several different application classes call methods of the complex package, then any modifications made to the package will necessitate a complete review of all these classes.

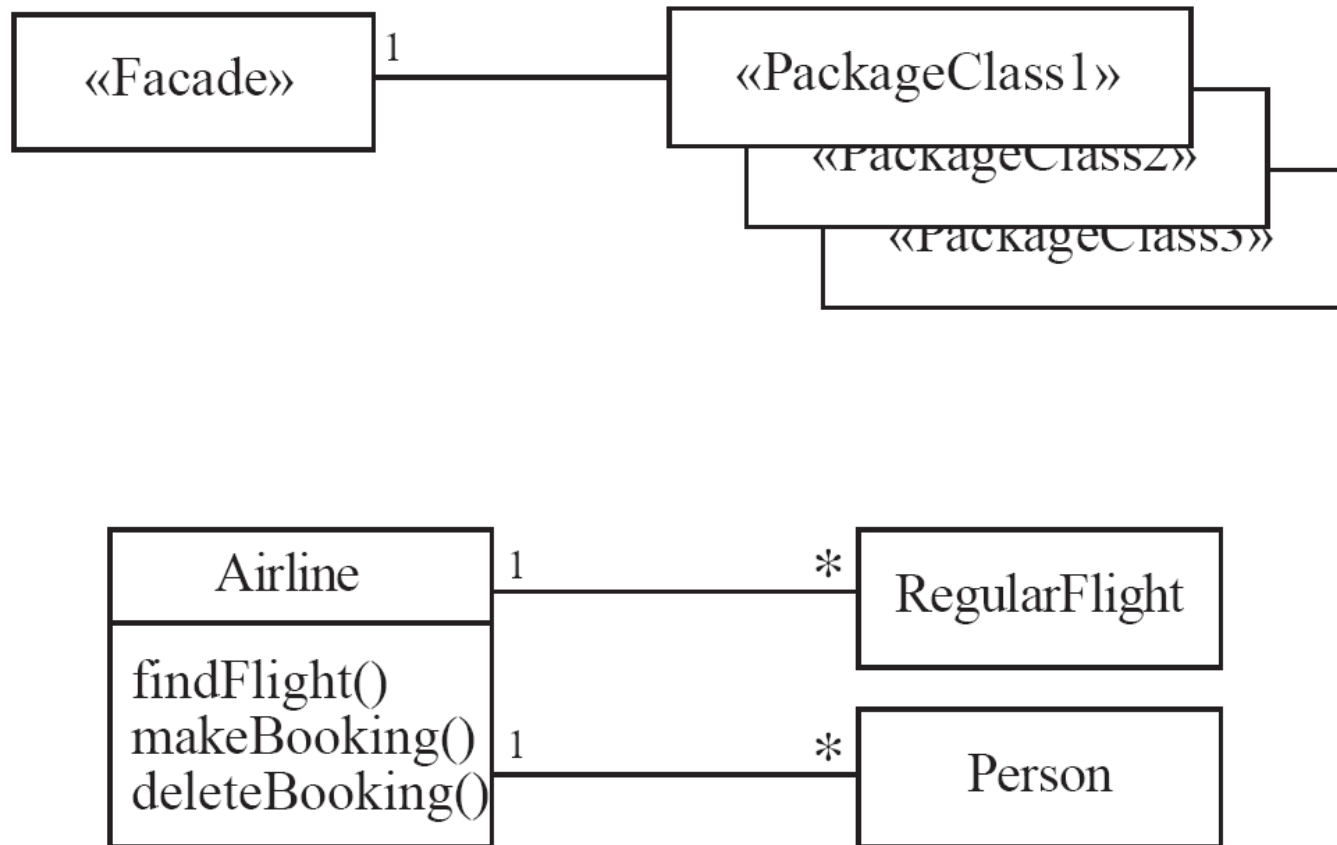
- **References:**

- Façade is one of the GoF patterns



# Façade

- *Solution:*



# 6.10 The Immutable Pattern

- **Context:**
  - An immutable object is an object that has a state that never changes after creation
- **Problem:**
  - How do you create a class whose instances are immutable?
- **Forces:**
  - There must be no loopholes that would allow ‘illegal’ modification of an immutable object
- **Solution:**
  - Ensure that the constructor of the immutable class is the *only* place where the values of instance variables are set or modified.
  - Instance methods which access properties must not have side effects.
  - If a method that would otherwise modify an instance variable is required, then it has to return a *new* instance of the class.
- **References:**
  - This pattern was introduced in [Gra02]

# 6.11 The Read-only Interface Pattern

- *Context:*

- You sometimes want certain privileged classes to be able to modify attributes of objects that are otherwise immutable

- *Problem:*

- How do you create a situation where some classes see a class as read-only whereas others are able to make modifications?

- *Forces:*

- Restricting access by using the **public**, **protected** and **private** keywords is not adequately selective.

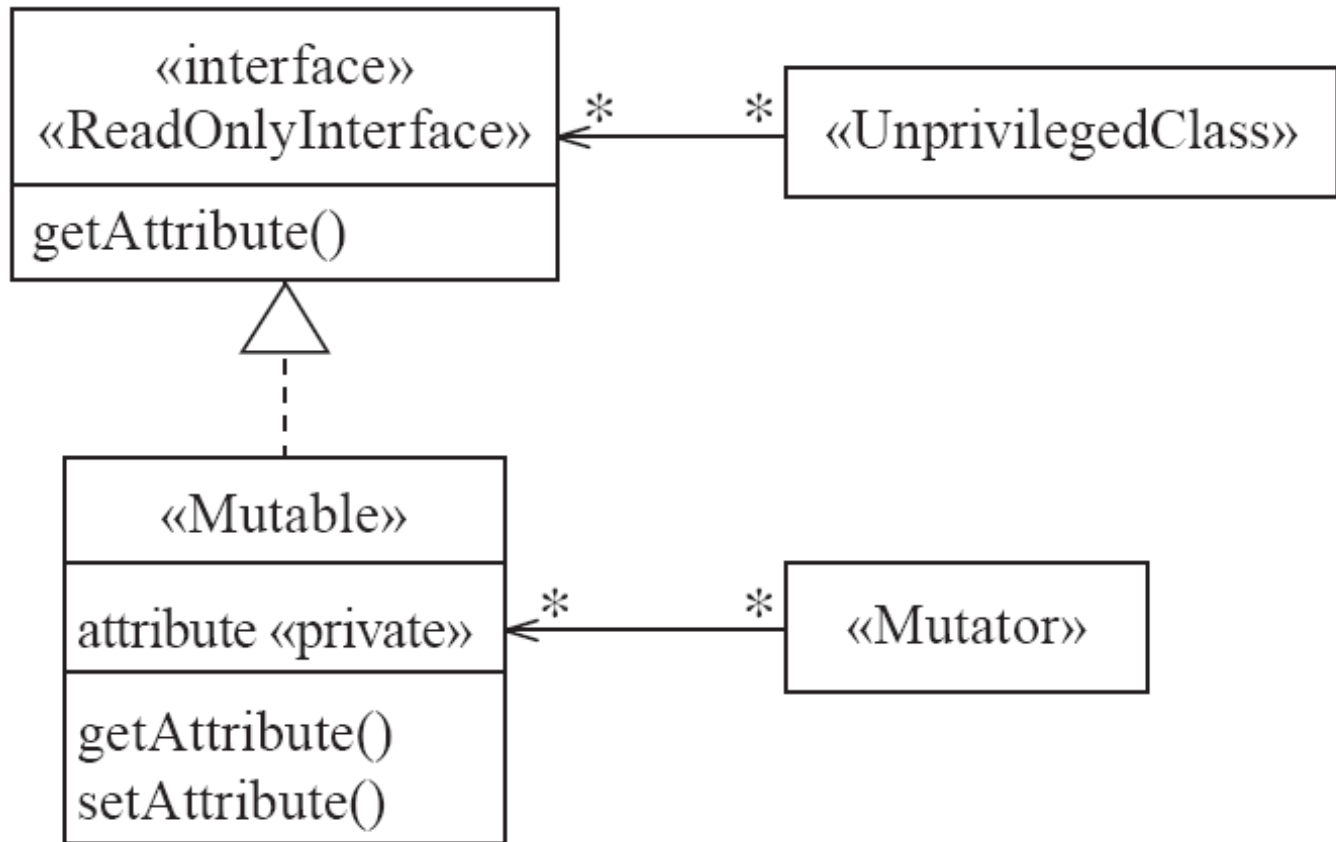
- Making access **public** makes it public for both reading and writing

- *References:*

- This pattern was introduced in [Gra02]

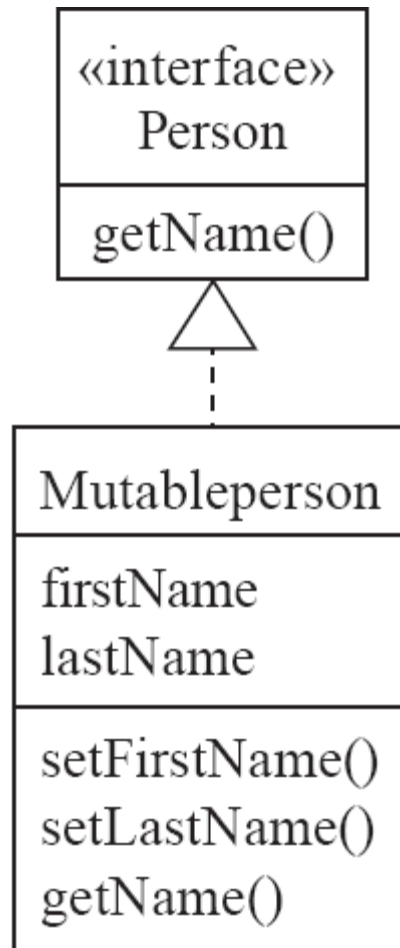
# Read-only Interface

- *Solution:*



# Read-only Interface

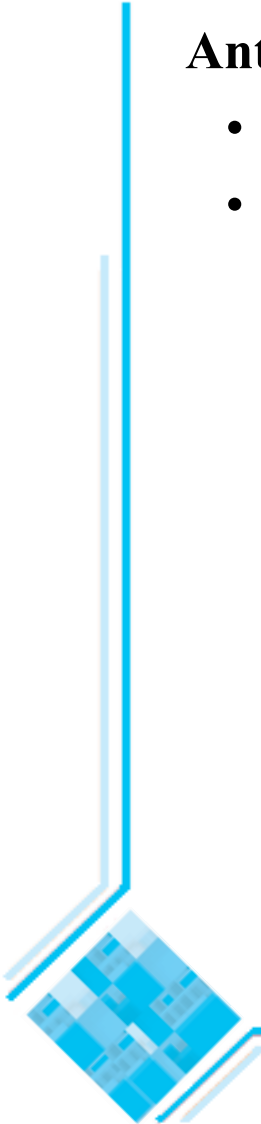
**Example:**



# Read-only Interface

## Antipatterns:

- Make the read-only class a *subclass* of the «Mutable» class
- Override all methods that modify properties
  - such that they throw an exception



# 6.12 The Proxy Pattern

- **Context:**

- Often, it is time-consuming and complicated to create instances of a class (*heavyweight* classes).
- There is a time delay and a complex mechanism involved in creating the object in memory

- **Problem:**

- How to reduce the need to create instances of a heavyweight class?

- **Forces:**

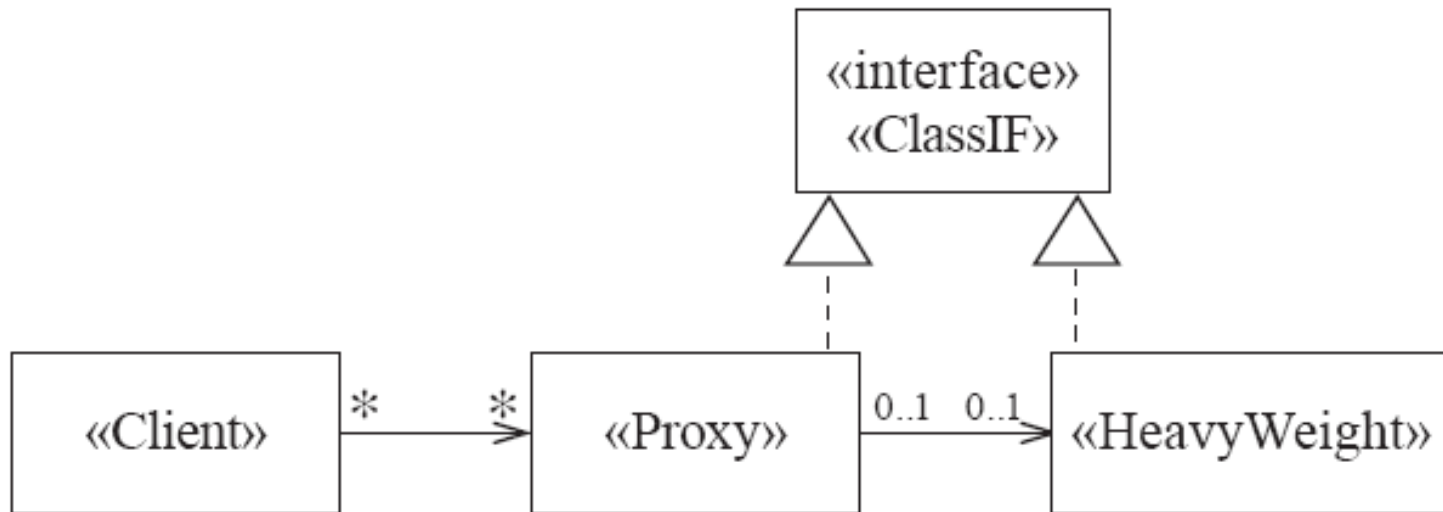
- We want all the objects in a domain model to be available for programs to use when they execute a system's various responsibilities.
- It is also important for many objects to persist from run to run of the same program

- **References:**

- Proxy is one of the GoF patterns

# Proxy

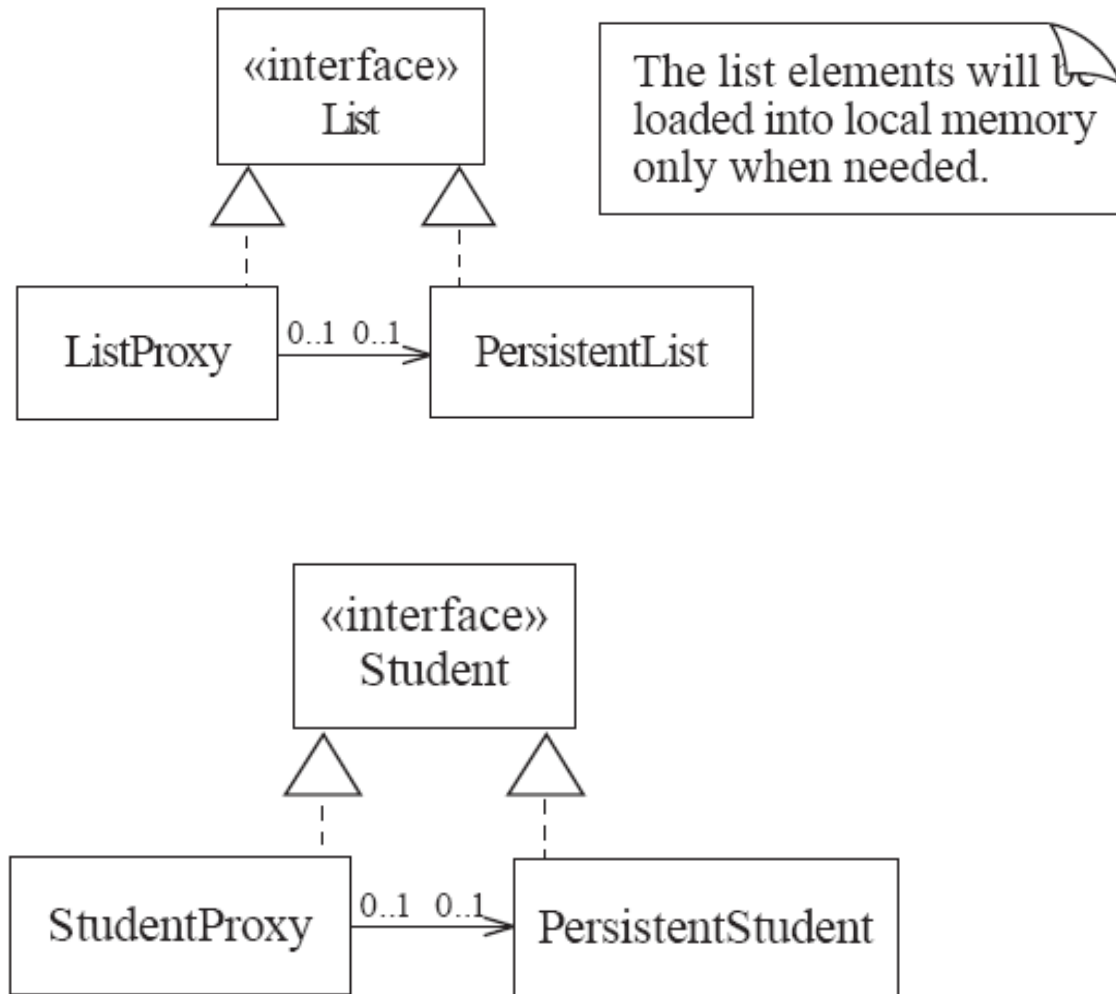
- *Solution:*





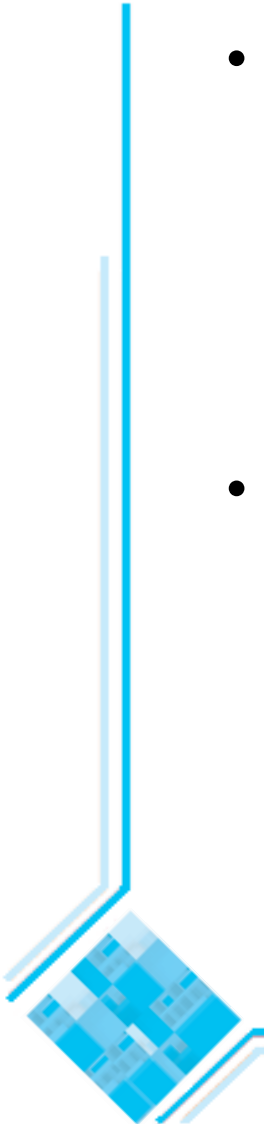
# Proxy

## Examples:



# Final remarks

- **Patterns are not a panacea:**
  - Whenever you see an indication that a pattern should be applied, you might be tempted to blindly apply the pattern.
  - This can lead to unwise design decisions .
- *Resolution:*
  - *Always understand in depth the forces that need to be balanced, and when other patterns better balance the forces.*
  - *Make sure you justify each design decision carefully.*



# References

- [GoF94] E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994. (The four authors of this book are referred to as the “Gang of Four”, frequently abbreviated as “GoF”)
- [EP00] H.-E. Eriksson and M. Penker, *The Business Modeling with UML: Business Patterns at Work*, Wiley, 2000.
- [Gra02] M. Grand, *The Patterns in Java Volume 1: A Catalog of Reusable Design Patterns Illustrated with UML*, 2<sup>nd</sup> edition, Wiley, 1999.
- [LL05] T. Lethbridge and R. Laganriere, *Object-Oriented Software Engineering – Practical Software Development using UML and Java*, 2<sup>nd</sup> edition, McGraw-Hill, 2005
- [Rum91] J. Rumbaugh et al., *Object-Oriented Modeling and Design*, Prentice Hall, 1991.
- [Nik16] I. Nikolov. *Scala Design Patterns*, Packt Publishing, 2016.